MCA PROJECT REPORT

Title: Employee Management API

Student Name : Ajay Kumar
Student ID : 10976147
Enrollment Number : 24DEOJULYMCA00126
Semester : MCA 3rd Semester
University : Kurukshetra University
Academic Year : 2024 - 2026

NOTE: The link to the project files is given below. Click on this link and you will be taken to a google drive link that contains the zip file of the project.

After downloading and extracting the project user can find instructions in the Readme.ms file to run this project on their local systems.

Link to the Project files:

https://drive.google.com/drive/folders/15OwUnyL7avfZiJleNRLzET_j4K25kPhu?usp=drive_link

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, and all those who supported me during the completion of this project.

PROJECT SUMMARY / ABSTRACT

Title: Employee Management API (RESTful Web API with Authentication & Role-Based Authorization)

This project aims to develop a secure and maintainable RESTful API for managing employee data with role-based access control.
It addresses common issues in manual employee record management such as data inconsistency, lack of security, and poor reporting.

The project uses Clean Architecture with ASP.NET Core, Entity Framework Core, SQL Server, JWT authentication, and role-based authorization.
Filtering, sorting, pagination, soft delete, and stored-procedure-based reporting are implemented.

Hardware and software requirements, testing approach, limitations, and future enhancements are discussed in detail.

STATEMENT OF THE PROBLEM

Organizations (small to large) maintain employee data such as personal details, department, designation, salary, hire date, and contact information. In many cases, the employee record lifecycle (creation, updates, deactivation, and reporting) is handled through manual processes or semi-structured tools such as spreadsheets. These approaches are easy to start with, but they create recurring operational problems:

- Inconsistent and duplicated data entries (same employee recorded multiple times or with variations)
- Weak validation and no standard data format across users
- Poor auditability (unclear who changed what and when)
- Limited access control (anyone with the file can view/edit sensitive fields like salary)
- Slow searching and reporting when the dataset grows.
- Difficulty integrating employee information with other software systems

From a software engineering perspective, the core problem is to create a centralized and secure system that can:

- Store employee data in a structured relational database
- Provide controlled access based on user roles (e.g., Admin vs HR vs Employee)
- Support modern “data listing” needs (search, filters, sorting, pagination)
- Support safe deactivation of records (soft delete) to prevent accidental data loss
- Support authentication and token management suitable for real applications (JWT + refresh tokens)

This project addresses the above by developing a RESTful Employee Management API with authentication, role-based authorization, and database-backed persistence using SQL Server.

Why this topic was chosen:

Industry relevance: Employee data management is a standard module in HRMS/ERP systems and is widely applicable to organizations of different sizes.

Alignment with syllabus: It allows implementation of database concepts (RDBMS, stored procedures), backend development, security concepts, and SDLC practices.

Hands-on demonstration of backend engineering: The project provides scope to implement and document:

- REST API design
- Layered architecture / Clean Architecture
- Authentication & authorization
- ORM (Entity Framework Core) and migrations
- Role-based access control (RBAC)
- Soft delete and audit-style timestamps
- Pagination/sorting/filtering patterns
- Integration of stored procedure for a report-like query

Practical learning value: The project is small enough to implement end-to-end, yet realistic enough to demonstrate maintainability and security requirements.

OBJECTIVE AND SCOPE

The objective is not only to create endpoints, but to develop a maintainable and extensible codebase that follows standard enterprise practices: separation of concerns, clean data access, security controls, and documentation/testing readiness.

Scope (what the project covers)

- Employee CRUD operations
- Create new employee record (Admin only)
- View employee list and employee details (authorized users)
- Update employee record (Admin only)
- Soft delete employee record (Admin only)

Employee listing capabilities

- Filtering by department, position, salary range, hire date range
- Sorting by selected fields (ascending/descending)
- Pagination with page number and page size

Security features

- JWT access tokens for API authentication
- Refresh tokens stored in the database for session continuity
- Role-based authorization using ASP.NET Identity roles (`Admin`, `HR`, `Employee`)
- Authorization policies (including an example of custom requirement handler)

Integration features

Google OAuth login endpoints (for external login flow)

A report-style endpoint calling a SQL Server stored procedure (`GetHighSalaryEmployees`)

Support utilities

- Swagger/OpenAPI documentation for discoverability and manual testing
- CORS configuration (for future frontend integration)
- A custom rate limiting middleware (attribute-driven)
- Bulk CSV upload endpoint to insert multiple employees

Out of scope (not implemented / not primary focus)

- Full UI frontend (this is a backend API project; UI can be added later)
- Advanced HR workflows like payroll calculation, attendance, leave approvals, reimbursements, etc.
- Advanced auditing (per-field audit logs), multi-tenancy, and enterprise SSO
- Sophisticated CSV validation/rollback pipelines (current bulk upload is simple)

SYSTEM PLANNING (PERT Chart)

The project plan was prepared using a PERT (Program Evaluation and Review Technique) approach to identify activities, dependencies, and estimated completion time.

Major activities were broken into phases:

- Requirement Analysis → System Design → Database Design → API Development → Security (JWT/Identity) → Integration (stored procedure/bulk upload) → Testing → Documentation.

Dependencies considered in planning:

- Database schema/migrations must be finalized before CRUD and reporting endpoints,
  and authentication/authorization must be completed before validating role-restricted endpoints.

Milestones were tracked as:

- (M1) Requirements freeze, (M2) DB + migrations ready, (M3) Auth module complete, (M4) Employee module complete, (M5) Integration/reporting ready, (M6) Testing sign-off, (M7) Final report & submission.
- PERT helped ensure parallel work where possible (e.g., documentation + API testing alongside implementation) while respecting critical-path tasks (auth + DB readiness).

Process Logic of each Module:

- The system is organized into modules (Auth, Employee, Cross-cutting). Each request follows a pipeline: request validation → authentication/authorization → business processing → database read/write → response DTO. Auth handles token generation and refresh rotation; Employee module handles filtering/sorting/pagination and soft delete logic; cross-cutting concerns include Swagger, CORS, and rate limiting.

Methodology adopted, implementation, hardware/software details:

- The project follows an SDLC flow with layered architecture (API/Application/Domain/Infrastructure). Implementation is done in .NET 9 with SQL Server using EF Core migrations, and verified using Swagger/Postman and database inspection tools.

System Maintenance & Evaluation:

- Maintenance activities include applying new migrations, updating role/policy rules, rotating secrets, and monitoring logs/errors. Evaluation is based on correctness of endpoints, security enforcement (401/403 cases), query accuracy (filters/pagination), and database integrity (soft delete + refresh token lifecycle).

Cost and Benefit Analysis:

- Cost includes development time, tooling, and database setup/hosting. Benefits include centralized employee data management, faster search/reporting, improved security through RBAC, reduced manual errors, and an extensible backend that can later support UI and additional HR features.

Detailed Life Cycle of the Project:

- The life cycle covers requirement gathering, design (API + database), implementation, testing, deployment readiness, and documentation.

Feedback from testing is used to refine endpoints, validations, and role restrictions before final submission.

- ERD, DFD: ERD models key entities (User–RefreshToken relationship and Employee entity).
  DFDs describe flows for login/token issuance, token refresh, employee CRUD, list/search, and high-salary reporting.

Input and Output Screen Design (if applicable):

- As an API project, "screens" map to Swagger/Postman request forms.
  Inputs are JSON bodies, query parameters, and CSV upload; outputs are JSON responses and status codes.

Process involved:

- Client sends request → API validates/authenticates → applies role policy → performs DB operation (EF Core / stored procedure) → returns structured response with appropriate HTTP status.

Methodology used for testing:

- Testing includes functional API tests (positive/negative), authorization tests (role-based access), integration tests for DB operations, and validation of stored procedure results.
  Results are recorded with request/response samples and screenshots.

Test Report / Printout of reports / Printout of code sheet:

- The report can include Swagger/Postman screenshots, sample JSON responses, database snapshots (tables for Employees/RefreshTokens), and selected code printouts for core modules (controllers, DbContext, seeder, repository).

User/Operational Manual (security aspects, access rights, back up, controls, etc.):

- Manual includes steps to run migrations, start the API, obtain tokens, and call endpoints. Security notes include role-based access (Admin restrictions), token usage, refresh token handling, and suggested backups (SQL Server database backup/restore) plus controls (rate limiting, secret rotation).

THEORETICAL BACKGROUND

The project is based on REST architecture, JWT authentication, role-based authorization, and layered software design principles.

SYSTEM ANALYSIS & DESIGN

The system is designed using Clean Architecture separating API, Application, Domain, and Infrastructure layers.

METHODOLOGY & IMPLEMENTATION

The project follows requirement analysis, system design, implementation, testing, and documentation phases.

HARDWARE & SOFTWARE REQUIREMENTS

Hardware: 8GB RAM, 64-bit CPU
Software: Windows 10/11, .NET SDK 9.x, SQL Server, Visual Studio

TESTING & EVALUATION

Testing is performed using Swagger UI, Postman, and database verification.

CONCLUSION

The project successfully demonstrates a secure, scalable, and maintainable employee management backend system.

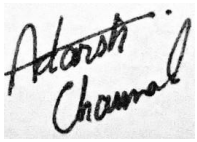
Annexure A

CERTIFICATE FROM GUIDE (Format)

This is to certify that this project entitled " Employee Management API " submitted in partial fulfillment of the degree of MASTER IN COMPUTER APPLICATION (MCA) to the Kurukshetra University done by Mr. AJAY KUMAR , Roll No. 10976147 is an authentic work carried out by him under my guidance. The matter embodied in this project work has not been submitted earlier for award of any degree or diploma to the best of my knowledge and belief.



Signature of the student



Signature of the Guide

Date: 29th Jan 2026